Lesson 3

Binary Images

Contents

1. Basic Image Thresholding
2. Colour Thresholding and HSV
3. Morphological Operations

Useful Links

- OpenCV Documentation: https://docs.opencv.org/4.x/
- OpenCV Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- Numpy References: https://numpy.org/doc/1.23/reference/index.html#reference

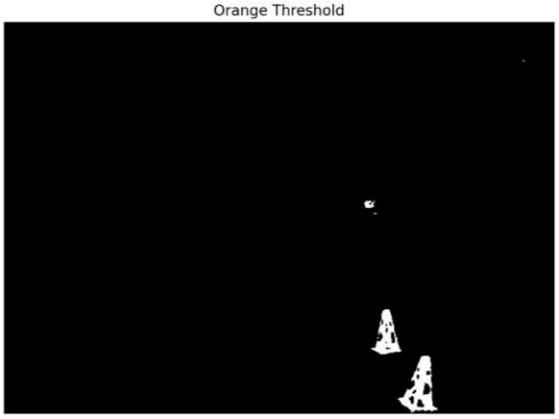
Overview

Binary images are an important concept in computer vision that involves an image consisting only of two colours, usually black and white. If done well, a binary image can completely isolate the parts of an image that we are interested in from the areas we are not.

One way of creating a binary image is through the use of a threshold. A threshold can be used to either find certain colours within an image we may be interested in or search for different levels of light intensity. Imagine we are trying to create a computer program that can find the cones in the image of the backyard below. We know that the cones are orange, so we can apply a threshold that only highlights pixels in the image that fall within the orange colour range. Doing so results in the threshold image on the right. The threshold contains significantly less noise and it becomes far easier to locate cones.

Original

Orange Threshold



Basic Thresholding

The most basic form of thresholding in OpenCV involves thresholding for pixel values that have an intensity above (or below) a set value. OpenCV provides us with two different functions to achieve this, *cv.threshold()* and *cv.adaptiveThreshold()*. In order to use either one of these thresholding functions effectively, we must pass in a grayscale image as an argument instead of the default BGR image returned by *cv.imread()*.

Pixels in a grayscale image have values ranging from 0 to 255 and represent the intensity of colour in a pixel. To convert an image from BGR to grayscale, use the *cv.cvtColor()* function seen in lesson one, or pass the *cv.IMREAD_GRAYSCALE* flag to *cv.imread()*.

```
img = cv.imread("image_path", cv.IMREAD_GRAYSCALE)      # option 1
gray =cv.cvtColor(image, cv.COLOR_BGR2GRAY)             # option 2
```

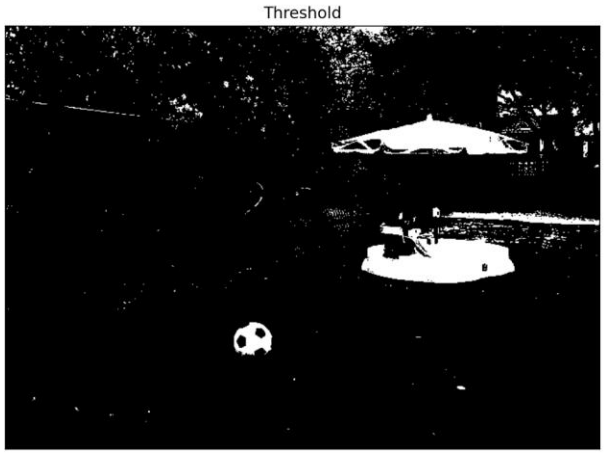
cv.threshold(image: cv.Mat, thresh_val, max_val, type) -> cv.Mat

- Returns a thresholded image. The type argument specifies the appearance of the returned threshold. *cv.THRESH_BINARY* is the most common. Check this link for other types of thresholding: https://docs.opencv.org/4.x/d7/d1b/group__imgproc__misc.html#gaa9e58d2860d4afa658ef70a9b1115576
- If using *cv.THRESH_BINARY*, the returned threshold image contains pixels that fall within the range of thresh_val to max_val.

cv.adaptiveThreshold(image, maxVal, method, type, blocksize, C)

- Better suited than *cv.threshold()* where lighting varies within an image. The intensity of a pixel is compared locally to its neighbouring pixels in a similar way to how a kernel is used. The blocksize argument specifies the size of the neighbourhood.
- Type is the same argument for *cv.threshold()*.
- Two statistical methods are available to use to compare a pixel intensity to its neighbours:
  - *cv.ADAPTIVE_THRESH_MEAN_C*
  - *cv.ADAPTIVE_THRESH_GAUSSIAN_C*

The below images are the result of converting the garden image seen earlier to grayscale and then running it through *cv.threshold()* with a thresh_val of 200.



Task 1: Creating an Intensity Threshold

1. Create a new python file and import the OpenCV library.

2. Find and download a video if you do not have access to a webcam.

3. Create a capture object with the video or using your webcam and start reading through the video frames using a while True loop. Make sure to not set the delay argument of *cv.waitKey()* too low.

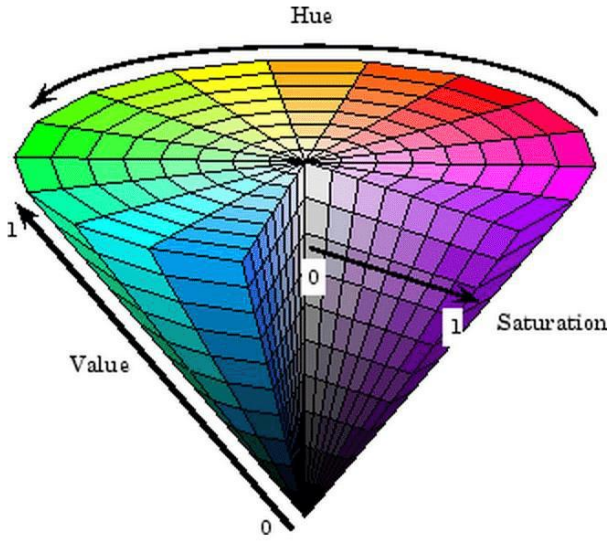
4. Convert the frame to grayscale and apply a basic threshold as well as an adaptive threshold and display them using *cv.imshow()*. What differences can you note between the two?

Colour Thresholding and HSV

Often we may already know the colour of an object or area we are looking for and it can be more useful to threshold for that specific colour rather than thresholding image intensity. The *cv.inRange()* function allows us to do exactly that.

When thresholding for an object of a specific colour, we must consider changes in our environment that can alter the appearance of an object to a camera. A common example is a change in lighting. Depending on the amount of light reflecting off an object, the shade of the colour of the object can either darken or brighten (i.e. a yellow ball will have a darker shade of yellow if it is in a darker room).

Unfortunately, RGB values tend to change non-linearly due to changes in shading, making RGB thresholding a difficult and unfruitful task. Thankfully, other colour spaces such as Hue, Saturation and Value (HSV) exist that are far easier to threshold.



Hue is a number between 0 and 360 which represents the colour tone, saturation is a percentage representing the amount of white in a shade (0% full white, 100% no white) and the value represents the darkness of a shade (0% black, 100% full colour). Each colour has its own hue (i.e. blue = 240) with the saturation and value changing for changes in brightness of a shade. Other colour spaces such as CMYK and HSL exist all with their own trade-offs, however, HSV will have you covered in 95% of cases. When choosing a colour space, consider what assumptions you can make and what challenges you are currently facing.

Annoyingly, the HSV colour space in OpenCV does not follow [0-360, 0-100, 0-100] for HSV colourings, instead following [0-180, 0-255, 0-255]. This is due to the maximum value an 8-bit unsigned integer can hold. Despite this, the concept of HSV colourings still holds, with saturation and value values dictating the whiteness and darkness of a shade while the hue value simply needs to be halved to find the corresponding value in the 0-180 range.

cv.inRange(image: cv.Mat, min_val: any, max_val: any) -> cv.Mat

- Returns a binary image with pixels that fall entirely within the *min_val* and *max_val* range set to white with all other pixels set to black.
- The dimensions of *min_val* and *max_val* must match the number of colour channels in the image being passed, i.e. if *image* is a HSV image, *min_val* and *max_val* must be tuples with three elements each. If using CMYK, *min_val* and *max_val* must be tuples with four elements each.

Task 2: Find the Duck

If you do not have a webcam or access to a rubber duck, please use the image below instead.

1. Create a new python file and import the OpenCV library.

2. Create a capture object that is connected to a camera on your computer and write a while true loop that can iterate through the capture object's frames. If you do not have a webcam, read the image into a variable using *cv.imread()*.

3. Use *cv.inRange()* to create a threshold for the duck. Have **INSERT MIN** as the *min_val* and **INSERT MAX** as the *max_val* if using your webcam. If you are thresholding the giant duck image, have (15, 100, 100) as the *min_val* and (35, 255, 255) as the *max_val*.

4. Try changing the saturation and value for *min_val* and *max_val* and see what happens.

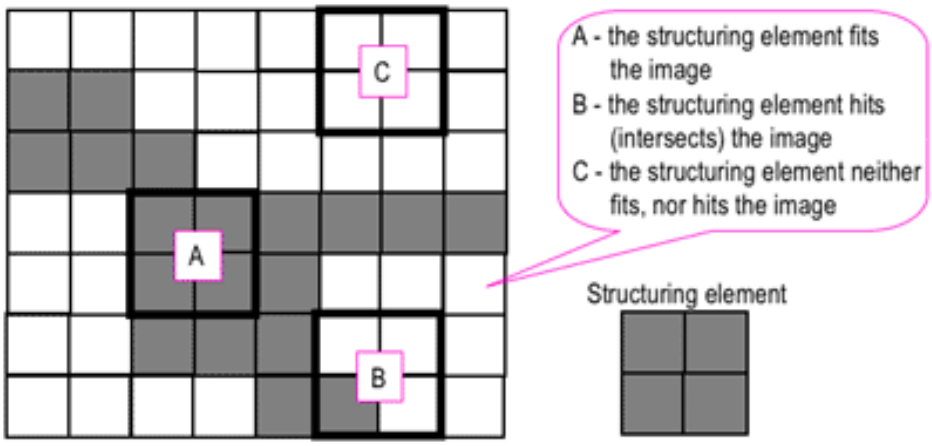
Morphological Operations

Unfortunately, thresholding operations do not always provide a perfect threshold of the object/area we are looking for. In many cases, our threshold can contain noise and sometimes miss parts of the object/area we are interested in. This can be due to many reasons, such as background objects having a similar colour to our object of interest or significant lighting conditions causing the HSV value of the interest object to fall outside the colour range we are looking for. Most of the time, threshold imperfections are negligible and can be navigated around using simple logic such as only looking for circles or using a region of interest (ROI). In the small number of times where we can't circumvent imperfect threshold images, Morphological Operations can provide us with an answer.

The above threshold attempts to threshold for dry sand. As can be seen, we are getting positive readings where there is no dry sand (noise) and the actual sand is appearing patchy. By applying some Morphological magic, we can both remove most of the noise and clean up the patchiness.

Although our threshold is still not perfect (you will very rarely get perfect images in computer vision), it is a considerable improvement from the threshold we had before.

Dilation, Erosion and the Kernel



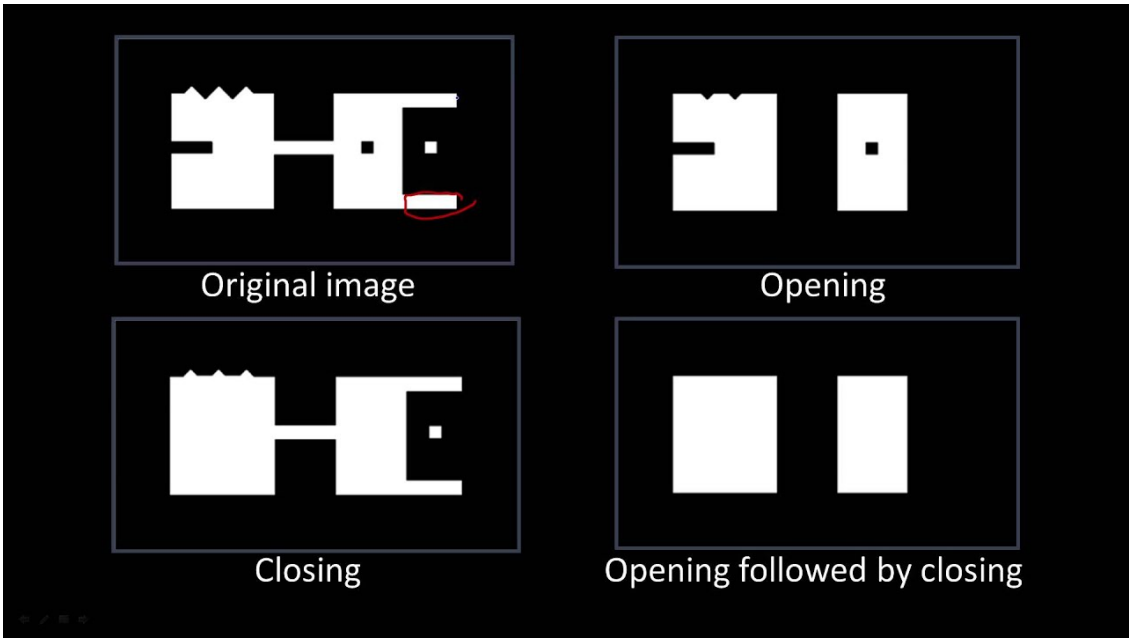
Hopefully you remember what a kernel is from lesson 2. If not, a kernel is simply a matrix with a specific shape that helps perform certain operations on an image. Morphological operations make use of a kernel, often called a structuring element. The values inside the structuring element do not matter as it is simply used to create a bounded area. The structuring element can take many geometric forms, such as squares, rectangles, circles, diamonds and crosses.

Dilation is an additive process that results in holes within a threshold being closed up, with the overall threshold thickening. Dilation causes the patchiness of the sand threshold on the previous page to close up. Dilation involves moving a structuring element throughout the pixels in an image with the centre pixel in the structuring element being our pixel of interest. If any coloured pixels are found anywhere within our structure element, the pixel of interest is also coloured in if it is not already so. In the above image, the A pixel will remain unchanged as it is already coloured, the B pixel will be coloured in as there is a coloured in pixel within the structuring element and the C pixel will remain uncoloured as no coloured pixels are inside the structuring element.

Erosion is a subtractive process that is effectively the opposite of dilation resulting in a thinned-out image with holes being opened. An erosion operation results in the pixel of interest (centre pixel) becoming uncoloured if not every pixel within the structuring element is coloured. In the above image, the A pixel would remain coloured as all pixels inside the structuring element are coloured, and the B pixel will become uncoloured (if it was already wasn't coloured) as not every pixel within the structuring element is coloured in with the C pixel remaining uncoloured for the same reason.

Both dilation and erosion operations result in a distorted image compared to the original passed in. In most cases, we want to preserve the overall look of the image while closing gaps in a threshold and removing noise. Dilation and erosion operations can be done back to back in order to counter the negative effects of both dilation and erosion. Dilation followed by erosion is known as opening and erosion followed by dilation is known as closing.

Closing is the operation done on the example image two pages prior and as the name suggests, closes holes and gaps in a threshold. Opening on the other hand does the opposite and can be good to remove unwanted links in an image as seen below.



cv.getStructuringElement(shape: int, size: tuple) -> np.ndarray

- The shape argument can take three different macros:
  - *cv.MORPH_CROSS*
  - *cv.MORPH_RECT*
  - *cv.MORPH_ELLIPSE*
- The values and what they specify in the size tuple change depending on what structure shape is being used. Please refer to the documentation to find what values need to be passed: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#gac2db39b56866583a95a5680313c314ad

cv.erode(image: cv.Mat, kernel: np.ndarray) -> cv.Mat

- Performs an erosion operation on the provided image using the kernel passed as an argument. Use *cv.getStructuringElement()* to get the appropriate kernel.

cv.dilate(image: cv.Mat, kernel: np.ndarray) -> cv.Mat

- Performs a dilation operation on the provided image using the kernel passed as an argument.

cv.morphologyEx(image: cv.Mat, operation: int, kernel) -> cv.Mat

- Performs a closing or opening operation depending on the operation flag using the provided kernel.
- Operation flags:
  - *cv.MORPH_OPEN*
  - *cv.MORPH_CLOSE*
- Optional argument of *iterations*= which allows you to easily complete as many opening/closing operations in succession on an image.
- Opening and closing operations are not the only combinations of morphological operations that can be done. Information other morphological operations can be found on the OpenCV tutorial page: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

Task 3: Cleaning the Beach

1. Download the image below and find an appropriate HSV colour range for the sand, either inspecting the pixels using Matplotlib or using the following website: https://pinetools.com/image-color-picker

   Keep in mind the differences between standard HSV and OpenCV HSV.

2. Create a threshold of the sand using *cv.inRange()*. Check if the threshold is good enough, if it is not, adjust the HSV range.

3. Once your threshold somewhat resembles the sand threshold on page 6, apply two opening operations using *cv.morphologyEx()*.